FULL-STACK TECHNICAL TEST

# Small Web Browser System Design & Execution Plan

Next.js · NestJS · MongoDB · TypeScript

**Objective**
Build a small database-backed web browser that supports address navigation, links, Back/Forward, persistent per-person history, full-content search, and publishing user-authored HTML.

**Engineering strategy**
Use a modular monolith, explicit navigation rules, backend-owned persistent state, strict API contracts, isolated HTML rendering, deterministic seed data, and automated tests for every critical user journey.

Prepared as an implementation blueprint for an approximately 8-hour focused build.

# 1. Architectural direction

The goal is not to build a miniature Chrome clone with every browser feature. The goal is to deliver the required behaviors correctly, with a small and understandable architecture.

**Recommended approach: modular monolith + domain-driven modules + explicit navigation state machine.**

| Principle | How to apply it | Why it matters |
|---|---|---|
| Domain-driven design | Organize NestJS around navigation, sites, search, history, and people. | Keeps business rules out of controllers and UI components. |
| Single source of truth | Backend owns persisted navigation state; frontend owns temporary UI state. | Prevents conflicting copies of history and current index. |
| Explicit state transitions | All navigation actions pass through one navigation service. | Makes Back, Forward, branching, and not-found behavior predictable. |
| Consistency | Persist session changes and visit records together where transactions are available. | Avoids partially recorded navigation operations. |
| Contract-first development | Define DTOs and API response types before frontend integration. | Reduces mismatches between Next.js and NestJS. |

**Why not microservices?** The scope is small, the deadline is short, and the application has one main database. NestJS modules provide separation without distributed deployment, network calls, or distributed consistency problems.

# 2. High-level system architecture

| Layer | Responsibilities | Suggested implementation |
|---|---|---|
| Presentation | Address bar, navigation buttons, content viewer, history, search, publisher. | Next.js App Router, React, TypeScript, Tailwind CSS. |
| Application/API | Validate requests, coordinate use cases, enforce navigation rules. | NestJS modular monolith, DTO validation, Swagger/OpenAPI. |
| Domain | Navigation transitions, address normalization, visit semantics, publishing rules. | Framework-independent TypeScript services and types. |
| Persistence | People, sites, browser sessions, navigation entries, visits. | MongoDB with Mongoose and appropriate indexes. |

Request flow: Next.js UI → typed REST API → NestJS controller → application/domain service → repository → MongoDB. Return a typed result and update the UI from the authoritative response.

# 3. Data model

Use separate records for the current navigable path and the immutable facts of what happened. This distinction is central to correct browser behavior.

## Collections

| Collection | Key fields | Purpose / rules |
|---|---|---|
| people | _id, name, createdAt | A selectable browsing identity. No authentication or roles are needed. |
| sites | _id, address, title, html, searchableText, authorId, createdAt, updatedAt | Published pages. Enforce a unique canonical address. searchableText is derived from sanitized page content. |
| browserSessions | _id, personId, entries[], currentIndex, version, updatedAt | Current browser navigation path for a person. Use one session per person for this assignment. |
| visits | _id, personId, address, siteId?, transition, outcome, visitedAt | Append-only record of navigation events. Back/Forward must not erase earlier visits. |

## Navigation entry example

```
interface NavigationEntry { entryId: string; destination: | { type: "site"; address: string } | { type: "search"; query: string; page: number }; status: "loaded" | "not-found"; scrollPosition: number; createdAt: Date; }
```

## Recommended indexes

• sites.address: unique index.
• sites title + searchableText: text index (or a dedicated search strategy if MongoDB text-index constraints require it).
• visits: compound index on personId and visitedAt descending.
• browserSessions.personId: unique index if there is exactly one session per person.

## Address normalization

Normalize and validate addresses on the backend before lookup or publishing. For example, Tidepool.ZZ and tidepool.zz should resolve to the same canonical address. Define allowed characters and the .zz address format explicitly. Do not treat arbitrary URLs as internal addresses.

## Search indexing

Extract plain text from HTML at publish time and store it as searchableText. Search title and extracted content, not raw markup. Recompute searchableText whenever a site is updated.

## Seed data

Use fixed people, sites, links, and timestamps. Use upserts and stable identifiers so running the seed command twice produces the same dataset without duplicates. Include approximately ten sites, several authors, broken links, repeat visits, and about an hour of activity across people.

# 4. Navigation engine: the critical subsystem

Do not implement Back, Forward, link navigation, and history jumps as unrelated frontend functions. Route all of them through one navigation domain service with explicit rules.

## Core navigation rules

| Action | Required behavior |
| --- | --- |
| Navigate to address / follow link | Resolve and normalize the address, truncate entries after currentIndex, append a new entry, and move currentIndex to the new final entry. |
| Back | Move currentIndex one step left if possible. Do not append a navigation entry or erase visit records. |
| Forward | Move currentIndex one step right if possible. Do not append a navigation entry. |
| New navigation after Back | Discard the forward portion of the active navigation path before appending the new entry. |
| Not found | Create a not-found navigation entry so Back returns to the previous page. |
| History jump | Move to the selected visit/destination according to a documented rule; do not silently corrupt the active path. |
| Refresh | Reload the current destination without creating an unintended duplicate history entry. Define whether a refresh event is recorded separately. |

## Branching example

Start with A → B → C → D. Go Back twice: current entry is B. Navigate to E. The active path becomes A → B → E; C and D are no longer available through Forward. The historical visit records for C and D remain.

## Pure navigation function

```
function navigate(entries, currentIndex, newEntry) { const retained = entries.slice(0, currentIndex + 1); retained.push(newEntry); return { entries: retained, currentIndex: retained.length - 1 }; }
```

## Derived button state

```
const canGoBack = currentIndex > 0; const canGoForward = currentIndex < entries.length - 1;
```

## Restore on return

Store scroll position on each navigation entry, not on the site. The same site may appear multiple times in history with different scroll positions. On Back/Forward, restore the selected entry's address, scroll position, and relevant view state. For the assignment, prioritize scroll restoration and document state that can be reliably preserved; do not claim a freshly reloaded iframe restores arbitrary JavaScript or form state.

## Concurrency and stale responses

Add a session version and use optimistic concurrency checks (expectedVersion). Protect the UI from out-of-order responses using request cancellation and/or a monotonically increasing navigation request identifier. A slower earlier response must never replace a newer destination.

## Persistence boundary

For a navigation that changes both the browser session and visit log, use a MongoDB transaction where supported so the two writes commit together. Transactions require a replica set or sharded cluster; configure a single-node replica set for local development if using transactions. Otherwise document and test a deliberate recovery strategy.

# 5. API contracts

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /people | List selectable people. |
| GET | /sites/:address | Resolve a canonical address. |
| POST | /sites | Publish a page. |
| GET | /sites/search?q= | Search titles and page content. |
| GET | /browser/:personId | Load persisted browser session. |
| POST | /browser/:personId/navigate | Navigate to an address or destination. |
| POST | /browser/:personId/back | Move backward. |
| POST | /browser/:personId/forward | Move forward. |
| POST | /browser/:personId/history/:entryId | Jump to a history destination. |
| GET | /browser/:personId/history | Return chronological visit records. |

## Navigation request contract

```
interface NavigateRequest { destination: { type: "site"; address: string } | { type: "search"; query: string; page?: number }; source: "typed" | "link" | "history"; expectedVersion: number; idempotencyKey?: string; }
```

The backend must validate the requested transition against the persisted session. Avoid endpoints that accept an arbitrary full browser state from the client and overwrite the database.

# 6. Frontend state ownership

| State | Owner | Notes |
|---|---|---|
| Selected person | Application/UI state | Load that person's persisted browser session when selection changes. |
| Navigation entries and current index | Backend | Frontend renders the authoritative response; do not maintain independent copies in multiple components. |
| Address input before submit | Local React state | Draft text can differ from the currently displayed address. |
| Loading / pending request | Request/UI state | Show progress and prevent stale responses from winning. |
| Search query and results | Query state + TanStack Query | Preserve query/page in a search navigation destination so Back can restore results. |
| Publisher form draft | Local React state | Backend validates and stores the published HTML. |

## Suggested frontend structure

```
frontend/src/ app/ features/ browser/ components/ hooks/ services/ types/ history/ search/ publisher/ lib/ api-client.ts query-client.ts validation.ts
```

Use TanStack Query for server state and React state or a small Zustand store for temporary UI state. Keep components feature-oriented. Generate or share API types from OpenAPI where practical.

# 7. Security model for published HTML

Treat every published page as hostile input. Never inject untrusted HTML directly into the Next.js application DOM.

## Defense-in-depth pipeline

| Stage | Controls |
|---|---|
| Input validation | Validate size limits, expected address format, and required fields. Reject malformed input. |
| Sanitization | Use a maintained HTML sanitizer with an explicit allowlist. Remove script elements, event-handler attributes, dangerous elements, and unsafe URL schemes. |
| Resource restrictions | Block external images, scripts, stylesheets, frames, and other network resources because hosted assets and real network access are out of scope. |
| Link handling | Allow only recognized internal .zz addresses. Reject javascript:, data:, and arbitrary external destinations. Route internal links through the navigation API. |
| Rendering isolation | Render sanitized HTML in a restrictive sandboxed iframe. Do not enable script execution or top-level navigation. |
| Browser policy | Use a restrictive Content Security Policy and test the rendered page in a real browser. |

## Iframe considerations

A sandbox without allow-scripts is a strong starting point. If parent-side scroll inspection or link interception requires allow-same-origin, carefully assess the origin boundary and ensure the untrusted document cannot execute scripts. Avoid casually combining allow-scripts and allow-same-origin. An opaque-origin iframe may require a different, carefully designed link/scroll communication approach.

## Security acceptance tests

Test script tags, inline event handlers, javascript: links, data: URLs, external resource references, forms, popups, attempts to navigate the parent, malformed HTML, and oversized HTML. Verify the actual iframe behavior, not only the sanitizer output.

# 8. Testing strategy

## Unit tests — navigation domain

• Navigate A → B makes B current.
• Back and Forward move the index correctly.
• Back at the first entry and Forward at the last entry are safe no-ops.
• Navigating after Back truncates the forward path.
• Missing addresses create not-found entries.
• Address normalization maps equivalent inputs to one canonical address.
• Search destinations can be restored.

## Integration tests — API and database

• Duplicate site addresses return a conflict.
• Each person receives an independent browser session.
• Back does not delete prior visit records.
• Search matches page body text, not just titles.
• Concurrent navigation cannot silently overwrite newer state.
• Failed transactions do not leave session and visit data inconsistent.
• Publishing sanitizes hostile HTML.

## End-to-end tests — Playwright

| Journey | Pass condition |
| --- | --- |
| Type an address | Correct page loads and the visit is recorded. |
| Follow four links | A → B → C → D works. |
| Back and Forward | Correct entry and scroll position are restored. |
| Branch navigation | A → B → C → Back → D removes C from forward path. |
| Search and return | Open a result, press Back, and restore query/results. |
| Publish and browse | Publish HTML, open its address, and follow its internal links. |
| Broken address | Not-found view appears; Back restores the previous site. |
| Switch person | Each person sees their own independent session/history. |
| Reload | Selected person, current destination, entries, and index restore. |

# 9. AI-assisted engineering workflow

Use AI tools as specialized assistants, not as autonomous architects. Keep one set of design documents as the source of truth and give the implementation agent bounded tasks with acceptance criteria.

| Role | Responsibility |
|---|---|
| Architect agent | Review schemas, contracts, navigation rules, security boundaries, and design trade-offs before implementation. |
| Implementation agent | Implement one approved module at a time; do not independently redesign shared contracts. |
| QA/security agent | Challenge edge cases, race conditions, data consistency, XSS, iframe isolation, and failure handling. |
| Reviewer agent | Compare the finished implementation against the original assignment and acceptance tests. |

## Recommended implementation sequence

| Order | Task | Acceptance criteria |
|---|---|---|
| 1 | Project setup and architecture docs | Apps start; module boundaries and contracts are agreed. |
| 2 | Schemas, indexes, deterministic seed | Seed can run twice without duplicates; expected data exists. |
| 3 | Navigation domain and backend APIs | Unit and integration tests prove transitions and persistence. |
| 4 | Next.js browser shell | Address bar, content, loading, error, Back/Forward use backend state. |
| 5 | Search and publishing | Content search works; hostile HTML is contained. |
| 6 | History and restoration | Per-person history and search-result return work. |
| 7 | Final tests and bug fixing | Critical Playwright journeys pass. |
| 8 | Walkthrough and verification | Commands work; narrated recording demonstrates required flows. |

## Project documents to create before major coding

```
docs/ SYSTEM_DESIGN.md DATABASE_SCHEMA.md NAVIGATION_RULES.md API_CONTRACTS.md SECURITY_MODEL.md
TEST_STRATEGY.md CLAUDE.md
```

# 10. AI coding-agent rules

Put these rules in CLAUDE.md or the equivalent project instruction file:

- Follow the approved system design and API contracts.
- Keep the backend a modular monolith.
- Keep navigation logic in one domain service.
- Backend owns persisted navigation state.
- Keep navigation entries separate from visit records.
- Normalize and validate addresses on the backend.
- Treat published HTML as untrusted; sanitize and isolate it.
- Implement one feature at a time; inspect existing code before editing.
- Avoid unnecessary rewrites and dependencies.
- Use strict TypeScript; avoid any.
- Handle loading, empty, error, and not-found states.
- Protect against stale asynchronous responses and concurrent updates.
- Make seed data deterministic and idempotent.
- Never claim tests passed unless they were actually run.
- Report changed files, decisions, tests, commands/results, limitations, and unresolved risks.

# 11. Eight-hour execution schedule

This is a time budget, not a guarantee. If time gets tight, reduce visual polish before sacrificing navigation correctness, HTML isolation, seed reproducibility, or the required walkthrough.

| Time | Work | Expected output |
|---|---|---|
| 0:00–0:35 | Architecture and project setup | Contracts, collections, modules, runnable app skeleton. |
| 0:35–1:20 | Database and deterministic seed | People, sites, links, visits, initial sessions. |
| 1:20–3:00 | Navigation domain and backend APIs | Back, Forward, branching, persistence, version checks. |
| 3:00–4:15 | Frontend browser shell | Address bar, viewer, navigation controls, loading/error states. |
| 4:15–5:20 | Search and publishing | Full-content search and secure HTML rendering. |
| 5:20–6:10 | History and restoration | Per-person history, search return, scroll restoration. |
| 6:10–7:25 | Testing and bug fixing | Unit, integration, and critical Playwright flows. |
| 7:25–8:00 | Recording and final verification | Run commands, clean demo data, narrated walkthrough. |

# 12. Recommended technology choices

| Layer | Choice | Reason |
|---|---|---|
| Frontend | Next.js + TypeScript + Tailwind CSS | Meets stack requirement; quick UI development. |
| Server state | TanStack Query | Caching, request lifecycle, cancellation, and invalidation. |
| Local UI state | React state or Zustand | Keep temporary state simple; avoid duplicating backend state. |
| Backend | NestJS + TypeScript | Modular structure, DTOs, validation, Swagger. |
| Database | MongoDB + Mongoose | Fits document-oriented site content and event records. |
| Search | MongoDB text index | Simple full-text search for the assignment's scope. |
| HTML security | Maintained sanitizer + sandboxed iframe + CSP | Defense in depth for untrusted published content. |
| Testing | Jest, Supertest, Playwright | Unit, API integration, and user-journey verification. |
| API docs | Swagger / OpenAPI | Makes contracts inspectable and frontend integration clearer. |
| Local infra | Docker Compose | Repeatable local setup; configure MongoDB replica set if transactions are used. |

# 13. Final engineering checklist

Before submission, verify:

- ■ Address normalization and uniqueness work.
- ■ Links navigate internally, including broken links.
- ■ Back and Forward are correct, including branching.
- ■ Scroll position is restored on return.
- ■ Search covers page body text.
- ■ Search results can be returned to after opening a result.
- ■ Publishing validates address and HTML.

- Untrusted HTML is isolated and cannot escape the application.
- Histories are independent per person.
- Visits remain recorded when navigation moves backward.
- Seed is deterministic and idempotent.
- Reload restores the current browser session.
- Loading, empty, not-found, and API-error states are understandable.
- Frontend and backend contracts agree.
- Critical automated tests pass.
- Both apps and the seed command are documented.
- Four-to-five-minute narrated walkthrough demonstrates all required flows.

# Final recommendation

**Build a small browser with correct semantics, not a large browser with fragile behavior.** The strongest submission will demonstrate deliberate trade-offs, a single authoritative navigation model, clear security boundaries, reproducible data, and tests that prove the required user journeys.